SPI-CODE · DESKTOP OPERATOR APP · DESIGN

SPI Code

A macOS and Windows desktop app for running the SPI program (System, Stack, Services) with GitHub Copilot as its only agent, built on the SPI domain already proven in keelson-rib-spi.

Operating SPI today takes four tools and an agent that knows nothing about the program. The `keelson-rib-spi` console in Keelson reads the program well. Its 23k lines and 464 tests cover failure attribution, incident folding, the upstream radar and template rollout. But it writes only to GitHub, and only through board buttons. Stack work happens in a terminal with `spi`, `kubectl` and `az`, and GitLab upstream work in `glab` or the browser. Rerun-when-deployable waits and workflow approval pauses are lost on restart. Setup means installing Bun, Keelson and seven command-line tools by hand.

SPI Code is one Electron app with three parts. The first is **@spi-code/core**, the domain extracted from keelson-rib-spi, which both the rib and the app use. The second is a **host process** that runs the sweeps and drives Copilot through `@github/copilot-sdk`, with SPI operator tools running in the host. It records every run, approval and write intent in SQLite, so a restart never loses a wait or repeats a write. The third is a **three-pane window**: the program tree on the left, a console view or an agent thread in the middle, and an inspector on the right. The app installs, versions and signs in the tools it needs, without Homebrew or a terminal.

SPI Code supports one agent provider and has no MCP server, CLI or plugin system. Threads come in two kinds. In an operate thread, Copilot has no shell and no file access, and every command it can cause is a typed tool with a declared tier. In a code thread, Copilot reads and edits files in an app-owned worktree, and each command it wants to run is approved on its own card and runs in an OS sandbox with no credentials. SPI Code itself makes the commits and the push, and the push is an approval card too. Fixes on a fork's main branch can still go to the Copilot coding agent. Linux is not a target. SPI Code does not replace the `spi` CLI or the fork workflows, and it never stands in for a human review gate. Its picture of the environment is no stronger than its evidence. Every view says when a source is unread, and a write that needs a deployable environment is refused while evidence is missing or in conflict.

COLORS IDENTIFY WHO ACTS

- **Operator · asks, approves, decides**
- **SPI Code · reads, plans, executes, records**
- **Copilot · reasons and proposes tool calls**
- **Outside systems · GitHub, GitLab, Azure, the cluster**
- **Dashed · installed or carried by SPI Code, not its own logic**

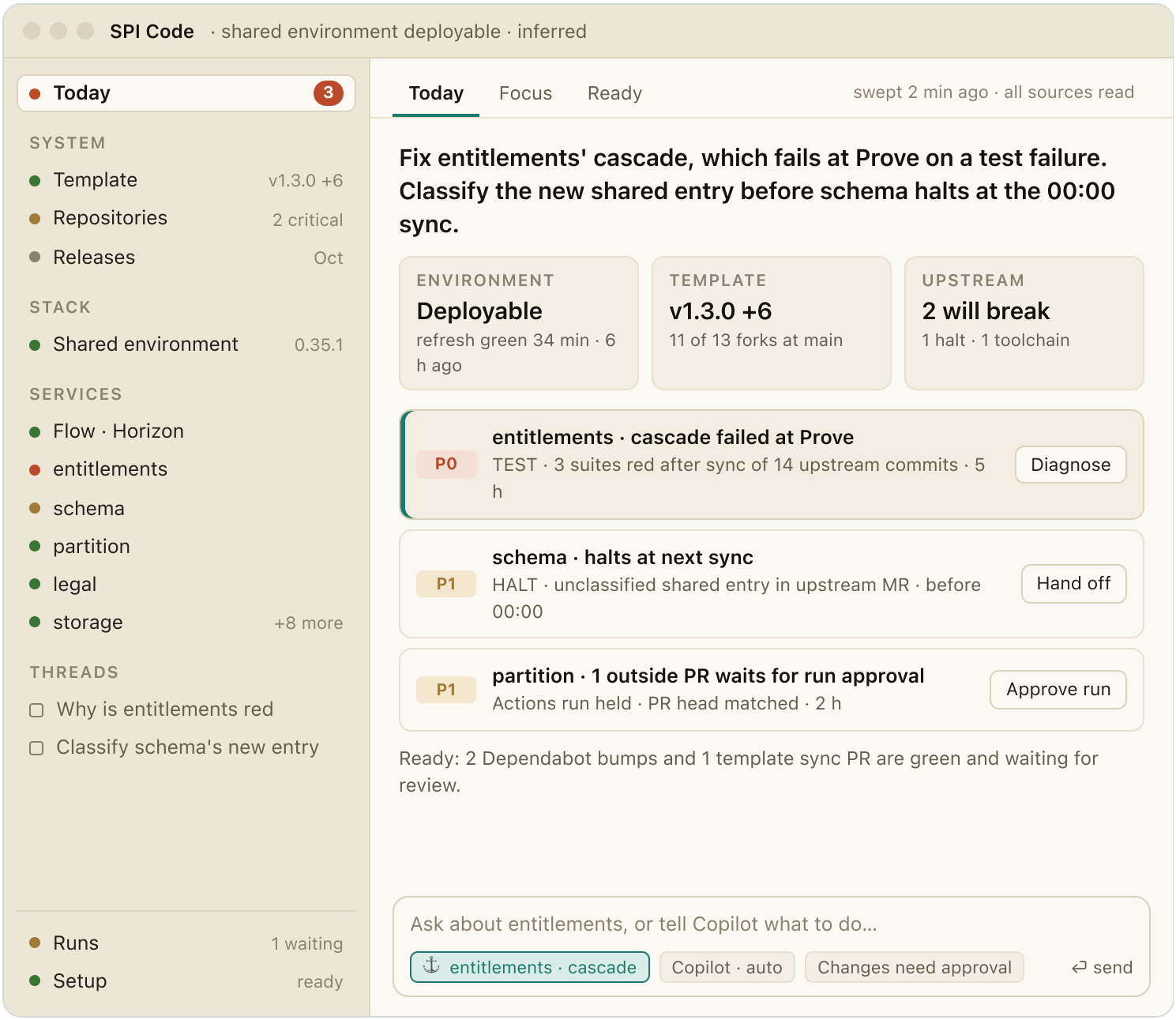
Program shape

The app is organized around the program's three nouns, not around repositories or chat sessions. Each noun is a branch of the left tree. Each object under a branch has a console view and can anchor agent threads. The rib console already builds six views (Today, Repositories, Environment, Flow, Horizon, Releases), and each one has a place in the tree.

BRANCH	OBJECTS	READ FROM	OPERATOR ACTS ON IT BY
Today	The program briefing: headline, facts, focus items, ready work	The whole model, through `buildBriefing`	Working the queue item by item from the inspector
System	The osdu-spi template, its release ladder and rollout, fork Repositories (issues, PRs, Dependabot, vulnerabilities, code scanning), monthly Releases	GitHub GraphQL and REST, template tags and compares, releases	Sync all forks, roll out, fleet release, release notes
Stack	The shared environment: running version, deployability, lifecycle runs, Flux state, pins	Lifecycle workflow runs from M2; `spi status --json`, Flux and pods from M6, labeled as live reads	Re-dispatch refresh or upgrade, reconcile, maintenance, service pin and reset
Services	Each service line (13 today) with its six stations, Flow across the fleet, Horizon (upstream radar, ports owed, library skew)	Fork Actions runs and PRs on GitHub; upstream heads, MRs and pipelines on GitLab	Approve runs, re-run, prepare and start upstream, cascade, merge Dependabot and template PRs, fix code in a worktree, hand a fix to the coding agent

Window

The layout follows T3 Code: a left bar, a middle panel and a right panel. The left bar is the program tree with live status dots, plus recent threads and active runs. The middle panel shows the selected object's console view, or an agent thread when one is open. The right panel starts with context tabs for whatever is selected (the inspector, a run, a code thread's changes, a PR), then Approvals and Log, always in that order. The UI mockups show thirteen screens and the component kit, all at one example moment.



Mockup in the light appearance. Names, counts and identifiers are examples. The composer always shows its anchor, so the operator knows what context Copilot starts from. A disabled action always says why, in place.

Threads anchor to objects

Every thread has an anchor: the program, a branch, a service line, the stack, a queue item or a PR. The anchor decides three things:

- Copilot's starting context: the object's `readItem` text (title, cause, effect, next action, related items) and the sweep time it came from.
- Where the thread is listed in the tree.
- Which operator tools are offered first. Every tool stays reachable.

Typing in the composer on a console view starts a thread anchored to whatever is selected.

Thread kinds

A thread's kind is set when it starts and shown in its header. An **operate thread** reads the program and proposes typed tools; Diagnose a failure runs as one. A **code thread** works in a worktree on one fork branch (see Code threads). The kind decides the session's `availableTools`, so Copilot in an operate thread has no file or command tool to call. When a thread already owns a piece of work, every view that shows the work opens that thread instead of offering to start another: the queue row reads "Fix awaiting push", and the inspector's primary action is "Open thread".

One approval, several places

Every approval card has one id and one state in the host's approval store: pending, binding failed, executing, applied, not applied, unknown, declined or invalidated. It shows where it was requested and in the Approvals tab, which is one inbox with the same count on every screen. The decision happens in one place per origin. A card requested in a thread is decided in that thread's composer strip, and its timeline entry and inbox row point there. A card requested by a button is decided on the card itself, in its run and in the inbox. A button never writes: it creates the card, and the card holds the decision.

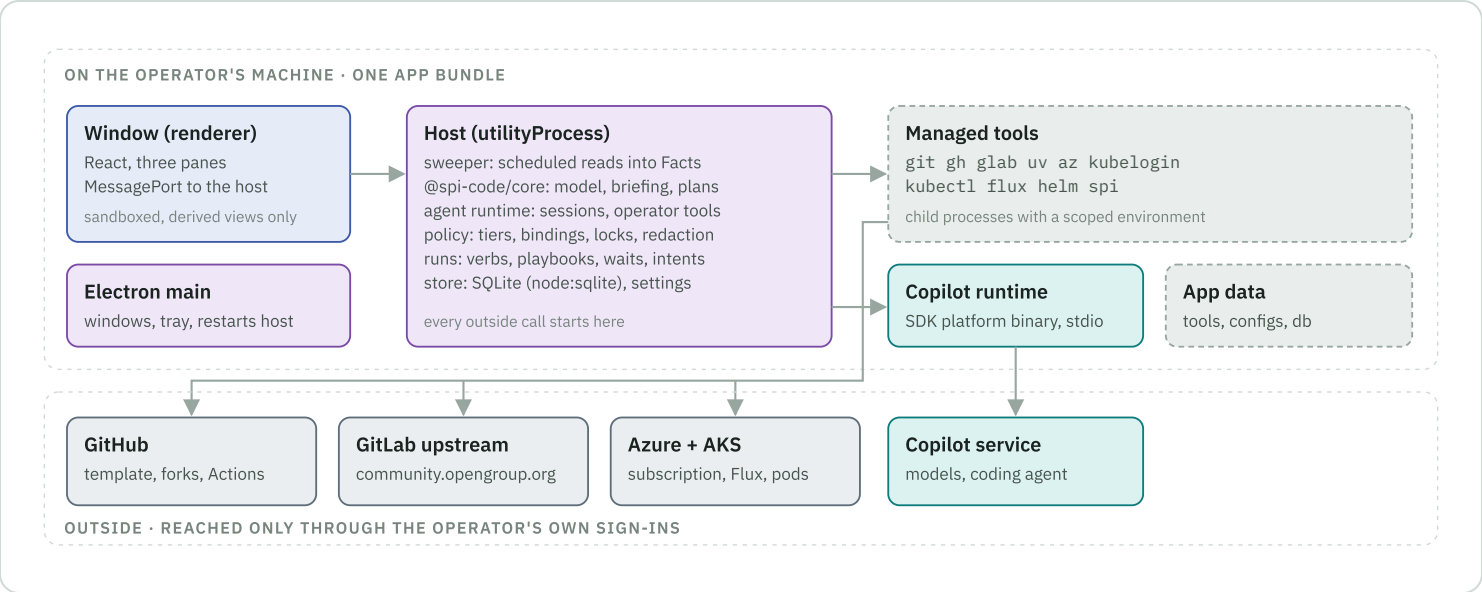
Window state

The window renders views of five host stores and keeps only the selection and each context's chosen tab.

STORE	HOLDS	FEEDS
SweepSnapshot	Facts, model, briefing and evidence envelopes from the latest sweep	The tree, every console view, evidence chips
ApprovalStore	Every card by id, with kind, tier, binding, typed name and state	Timeline cards, the composer strip, the Approvals inbox, Runs
IntentLedger	Runs, intents, per-target outcomes and lookups	Runs, outcome chips, the PR watch
WorktreeStore	Worktree, local branch, base and head, uncommitted and unpushed counts, push lease	The tree, the thread header, the Changes tab
ThreadStore	Thread kind, anchor, Copilot session and timeline	The timeline, the work log, the composer

Process model

SPI Code is Electron. Like T3 Code, it splits a thin main process from a separate backend, adjusted for an app that is never served remotely. The host runs as an Electron `utilityProcess` and talks to the window over a `MessagePort` with zod-validated messages. There is no local network port, so nothing else on the machine can reach the agent. M0 measures the Copilot runtime and `node:sqlite` inside a utility process on signed builds. If either misbehaves, the host becomes a Node child process started with `ELECTRON_RUN_AS_NODE`, as T3 Code runs it. Only the chosen model is built after M0. The message contract is the same either way.



The window never runs a command and never calls the network. The host pushes it derived view models (briefing, view rows, inspector text), never raw Facts. Every button becomes a host request that passes the same policy check as a Copilot tool call.

Closing the window leaves SPI Code in the menu bar on macOS and in the tray on Windows while any run or wait is open. Quitting with open runs asks first. Waits resume on the next launch, and a single-instance lock stops two hosts from sweeping or writing at once. If the host crashes, main restarts it once. The window then resubscribes from its last event cursor, so no run event is shown twice or skipped.

Copilot runtime

The host drives Copilot through `@github/copilot-sdk`, pinned at a version Keelson's provider already runs (1.0.26 today). The SDK ships its own runtime for each platform in `@github/copilot-sdk-<platform>`, so the app bundles no separate CLI. Packaging unpacks that platform package from the asar archive and starts it over stdio by explicit path. It never uses the SDK's in-process mode, which loads a native library into the host and conflicts with the hardened runtime. Each architecture gets its own build because only the matching platform package is installed.

Sessions are isolated from anything the operator has set up for Copilot elsewhere. The client runs in `empty` mode with a base directory under app data, which keeps the operator's `~/.copilot` MCP servers, agents, skills and instructions out of SPI Code. Empty mode also means the runtime never sees the operator's existing Copilot login. Instead, each session gets its token from a provider that reads `gh auth token`, so one gh sign-in covers both tools and a `gh auth switch` takes effect on the next session. That provider API is marked experimental, so M0 proves it with no client token: start, list models, stream a turn and refresh. If it falls short, the host passes the gh token at the client level instead, and drops the warm client when the credential changes, as Keelson does. The token stays in gh's keychain entry, and no Copilot credential is written under app data. `availableTools` lists every allowed tool by name: each SPI tool as `custom:<name>` and no built-in tool at all. Every SPI tool is registered with `defer: "never"`, because with no built-in tool search a deferred tool would be out of Copilot's reach. Copilot never gets a built-in shell, file, URL fetch, memory or subagent tool. In an operate thread it has no file or command tool at all, and when it needs a log or report it calls `read_artifact`, which reads only files the host itself saved for the current run. A code thread adds the host's worktree tools from appendix C: file reads, search and edits confined to the worktree, `run_command` and `commit`. `onPermissionRequest` denies every kind, so a built-in tool that ever slips through is refused rather than allowed.

SPI tools are registered with the SDK's permission prompt skipped, and the policy check runs inside each tool's handler. The handler is the only place that can hold the target's lock, re-read the binding and write the intent around the actual call. Buttons and playbooks call the same handler, so all three paths share one check. The check runs after the arguments pass the tool's zod schema and before any outside call. Any error in that check, or an answer it cannot read, fails closed. Model choice comes from the operator's Copilot plan, shown as a picker with `auto` as the default.

Each session starts with a system message seeded from the rib's Watch Officer prompt, followed by the anchor's context and its sweep time. The prompt's rules: read before acting, keep certain upstream changes apart from possible ones, never assume maintenance can be cleared, and name the exact next action. Appendix D lists the session lifecycle hazards Keelson's provider already handles; the host copies those fixes.

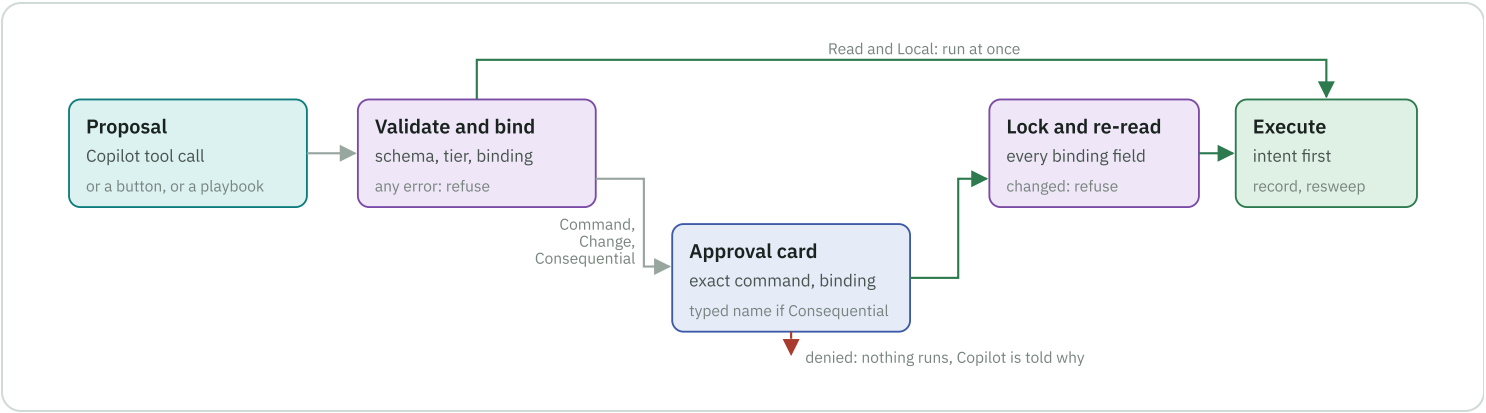
Authority and approvals

SPI Code can act in GitHub, GitLab, Azure and the cluster through typed tools, with the operator's own sign-ins. Each tool builds its own command from validated arguments, supports an explicit list of variants, and declares one tier. Outside a code thread's sandbox, no tool runs an arbitrary command. A button press, a playbook step and a Copilot tool call all go through the same check. Copilot proposes. Neither Copilot nor a playbook can approve a proposal.

TIER	RUNS WHEN	EXAMPLES
Read	At once, with no prompt. Logged, and redacted before the result reaches Copilot or the store.	Every `spi_*` read tool, `spi status --json`, `kubectl get` (not Secrets) and bounded logs, `flux get`, run logs, MR diffs, `az` show and list
Local	At once, inside a code thread's worktree only. Shown in the thread's work log.	Read, search and edit files in the worktree (paths checked after resolving links); commit to the thread's local `spi/*` branch
Command	After the operator approves the card, which shows the exact command and folder. Runs in the worktree sandbox. Code threads only.	Test runs, builds and formatters, such as `mvn -pl <module> test`
Change	After the operator approves the card. One card, one call.	Push a code thread's branch (typing the PR number when the push changes `.github/`), open a PR, re-run a cascade, re-run failed jobs, approve an outside PR's Actions run, merge a Dependabot or template PR into a protected branch once checks and reviews pass, dispatch template sync to one fork, prepare upstream, open an issue or upstream MR, assign the coding agent, comment
Consequential	Only after the operator types the target's name into the card. Never batched with another card.	Anything that touches the shared environment, deletes, or acts on many forks at once: sync all forks, roll out, start upstream, env refresh, upgrade or reset dispatch, maintenance set and clear, `spi reconcile` (plain, resume, refresh-images; `--suspend` is not offered), service pin and reset

TIER	RUNS WHEN	EXAMPLES
Not offered	No tool exists for it.	Commands outside a code thread's sandbox; `git push`, `gh`, `az`, `kubectl`, `spi`, `flux` or `helm` as commands; merging a cascade PR (gate 2 is a review on GitHub); submitting a PR review approval, squash-merging release or upstream branches, force push, repository secrets, reading Kubernetes Secrets, `info --show-secrets`, any guard bypass

Each Change and Consequential tool declares its binding: the facts that must still hold when it runs (appendix C). For a merge, those are the head SHA, base SHA, mergeability, required checks, review state and draft state. The merge itself is sent with `--match-head-commit`, so GitHub refuses it if the head moved after the check. For an environment upgrade, the binding is the declared version, maintenance state, no lifecycle run in flight, and the tenant, subscription and cluster the card names. A recovery tool binds the state it exists to fix. Clearing maintenance makes the same check the lifecycle workflows make before they clear it: no lifecycle run in flight, the deployed ref matches the declared stack version, and `spi status` reports the environment ready, with maintenance as its only blocker. It does not require the environment to read deployable, because clearing maintenance is what makes it deployable. A pending card does not expire. When the operator approves, the host takes the target's named lock, re-reads every binding field, and runs only if all of them still hold. Otherwise the card says which field changed and nothing runs. Named locks per fork and per environment serialize conflicting writes across threads, playbooks and buttons.



Every call, including a Read, is written to the run log with its arguments, result size and duration. Child processes never inherit `SPI_SKIP_GUARD` or another guard-bypass variable.

Evidence before a write

Every read result carries the same envelope: its source, when it was observed, whether it is complete, and whether it is fresh, stale, unread or failed. A source is stale when its last good read is older than twice its sweep interval, incomplete when a page or size limit cut the read short, failed when its latest read errored, and unread when no read has succeeded since the app started. Views show that envelope wherever a value is drawn. A write whose binding needs a deployable

environment is refused when deployability is unread or when sources disagree, for example when the inferred model says deployable but a live `spi status` says blocked. The card names the disagreement. Both values stay visible. Neither silently wins. A value drawn only from inferred sources says so wherever it is shown, including the top bar ("Deployable · inferred").

Redaction runs on every result before it reaches Copilot or SQLite. It strips tokens, keys, connection strings and kubeconfig credentials. Pod logs and `kubectl get` results have size limits, and Kubernetes Secrets have no reader.

Code threads

Some fixes have to land on `fork_integration` or in files the template owns, where the coding agent cannot work. A code thread makes those fixes in an app-owned worktree. It starts from a finding or an inspector action, checks out the fork at the head the finding read, and carries the finding as context. Each fork is cloned once under app data and shared by its worktrees.

Each code thread commits to its own local `spi/*` branch and names the remote branch it targets, for example `spi/fix-prove-418 → fork_integration`. One thread at a time holds the lease to push to a given remote branch. The thread header shows the branch, the target, and the base and head commits.

STEP	WHO ACTS	APPROVAL
Read, search and edit files	Copilot, through host tools confined to the worktree	None (Local)
Run a test or build	Copilot proposes; the sandbox runs it	One Command card per command
Commit to the local branch	SPI Code, at Copilot's request	None (Local); recorded in the work log
Push the branch	SPI Code, through `push_branch`	Change card; the PR number is typed when the push changes `.github/`
Re-run the cascade	SPI Code, through `rerun_cascade`	Its own Change card; a push does not re-run it
Review and merge the cascade PR	A reviewer on GitHub	Not offered in the app

After the push, the thread watches the PR at the pushed commit: each required check on that commit and the review gate 2 needs. When the checks pass, it links to the PR on GitHub for the review. The worktree is kept until its branch merges and nothing in it is unpushed, and while any thread, run or approval still uses it.

Command sandbox

Commands run under an OS sandbox: Seatbelt on macOS and a restricted token on Windows. If the sandbox cannot start, the thread offers no commands. Inside it, a command:

- can write the worktree checkout only. The clone's git folder, hooks and config are read-only to it, which is why SPI Code makes the commits.
- gets its own home folder and an allow-listed environment, with no `GH_TOKEN`, `GITHUB_TOKEN`, `AZURE_*`, `ARM_*` or `SSH_AUTH_SOCK`, and no credential helper.
- cannot read the keychain, `~/.ssh`, `~/.kube`, `~/.azure` or the gh config.

- is refused when it is `git push`, `gh`, `az`, `kubectl`, `spi`, `flux` or `helm`. Those actions go through SPI Code's own tools.

Network access stays on, because Maven downloads dependencies, so an approved command can send whatever the worktree holds. The card shows the exact command for the operator to judge, and Settings states the limit.

SPI Code's own git

Commit, status, diff and push run outside the sandbox. Each of those calls names the clone's git folder directly and never follows the worktree's `.git` file, which a sandboxed command can write. Each runs with hooks off and repository config ignored. `push_branch` pushes the exact commit from the fork clone to the remote URL SPI Code holds. The push is fast-forward only and is refused unless the remote is still at the commit the card names. Its binding is read when the card is made and again on approval (appendix C).

Terminal

Each worktree has a terminal drawer with two tabs. The first is a read-only view of Copilot's approved commands and their output. The second is the operator's own login shell in the worktree, outside the sandbox and labelled that way, for rebasing, resolving conflicts and committing by hand. SPI Code re-reads the worktree before it makes any card, so work done in that shell is picked up.

Runs and durability

Everything that does work is a run: a verb pressed in the inspector, a Copilot turn, a playbook, and a wait such as rerun-when-deployable. A run is a row in SQLite with an append-only event list. The window shows a run's log in the right panel and lists active runs and waits in the left bar. Each host write (a push, a re-run, a dispatch) gets a run. Commands and commits stay in their thread's work log.

Before any outside write, the host records an intent: the tool, its exact arguments, the binding it checked, and the facts that will identify the result, such as the dispatch time and actor or the PR number. After the call returns, the intent is closed with its result. If the app stops between the two, the run reopens as **outcome unknown**. Each write tool declares how to look up its own outcome, for example whether the PR is now merged, or the state of the workflow run the dispatch returned. The host runs that lookup before offering retry and never re-sends a write without one. The lookup ends in one of three states. Applied closes the run. Not applied allows retry, and is reached only on positive evidence that the write did not happen, never because time passed. Unknown keeps the run open, with retry disabled, until a lookup succeeds or the operator records that they checked by hand. Writes that create text on GitHub or GitLab (an issue, an MR, a comment) carry the intent id in a hidden marker, so the lookup finds them exactly. A write to several targets keeps a ledger with one outcome per target and never retries a target that is applied or unknown. A target whose last write by the same tool is unknown is held out of new fleet actions, and named, until the operator confirms the candidate run or not applied is proven. Appendix C gives each tool's lookup. The M4 issue carries the exact fields and the crash-test matrix for every write, and M4 starts from it.

Waits and verb or playbook approvals survive a restart. A wait resumes its condition check on the next sweep. A pending approval card shows again and binds when approved. A Copilot tool call does not survive a restart, because the SDK restores the conversation but not the in-flight call. That call is marked cancelled, and Copilot is told so on the thread's next turn. A command running in a code thread's sandbox stops with the host, and running it again takes a new card.

Playbooks

A playbook is a versioned state machine the host runs as one run. Each step has a stable id and is one of four kinds: a tool call, a Copilot turn with a narrowed tool set and a schema-checked result, an approval, or a wait on a model condition. Every transition is committed before the next step starts. On restart the host reads the last committed transition and continues from there, or reports outcome unknown when the step was a write. A run keeps the playbook version it started with. Playbooks are TypeScript in the app, not user-authored YAML. Version 1 ships two.

PLAYBOOK	STARTED FROM	STEPS
Diagnose a failure M3, cluster evidence M6	Any incident or failed lane in the inspector	Read the failed job, step and reason, annotations and logs, then the core's attribution, the image digest and the pin owner. From M6 it also reads Flux state, pods and jobs for the service. Copilot writes a finding that cites each tool result, names the cause, and names the verb that clears it. Read only.
Hand off a fix M4	An item whose next action is a code change on the fork's main branch: a port owed, a dependency, a failing test, a filter classification	Copilot drafts an issue with the cited evidence. After approval, the host opens it on the fork and assigns the Copilot coding agent. It then follows the PR, offers `approve_runs` when the PR's workflows wait for approval, and links the PR to the item.

The coding agent opens PRs against the default branch, so it does not handle `fork_integration` conflicts or workflow files the template owns. For those items the inspector offers a code thread instead of a handoff, and Diagnose suggests one as its next step. Drive a PR to green and Upgrade the stack follow version 1, built on the same runner (appendix E).

Setup and tools

The first launch opens Setup instead of Today. Setup lists each tool with one state, as T3 Code does for providers: checking, install, sign in, ready, or attention. Tools are grouped by the capability they unlock (Watch, Code, Operate), so Watch works before a JDK or any Azure tooling is installed. Probing a tool is always read-only. Installing or signing in is always a button the operator presses, and its output streams into a run log.

WATCH · READ THE PROGRAM			
Copilot	Ready	Runtime inside the app · uses the gh sign-in below	
git	Ready	Found on the login-shell PATH	
gh	Sign in	Installed by SPI Code · needs repo and workflow scopes	Sign in with browser
glab	Install	For community.opengroup.org · pinned release, checksum verified	Install
CODE · WORKTREES AND BUILDS			
Worktree folder	Ready	App data · each fork cloned once and shared by its worktrees	
JDK 17 · Maven	Install	Release archives, checksum verified · one shared Maven repository	Install
Command sandbox	Ready	Sandbox profile checked · code threads can run commands	
OPERATE · STACK AND AZURE			
uv · Python	Ready	Installed by SPI Code · Python managed by uv under app data	
az	Sign in	Installed with uv · k8s-configuration extension added · sign in to the stack's tenant	Sign in
spi	Attention	0.35.0 installed · the stack declares 0.35.1	Update
kubectl · kubelogin · flux · helm	Ready	Installed by SPI Code · app-scoped kubeconfig	

Single-binary tools (gh, glab, uv, kubectl, kubelogin, flux, helm) install from a manifest that records the version, download URL and SHA-256 for each OS and architecture. They go into the app's own `tools/` folder. uv provides everything Python. Its Python, tool and binary folders all live under app data. The Azure CLI installs as `uv tool install azure-cli --with pip`, because `az extension add` needs pip, and Setup then adds the `k8s-configuration` extension that spi's cluster guard falls back on. `spi` installs the wheel from the stack's GitHub release through

Microsoft's package feed proxy, at the version the stack declares in `ops/environments/shared.yaml`. Setup flags any mismatch, so a stack upgrade prompts a CLI update. Neither platform needs Homebrew, WinGet or administrator rights. The Code group installs JDK 17 and Maven from their release archives the same way and shares one Maven local repository across worktrees. git is the one tool taken from the system: Xcode command line tools on macOS, and PortableGit unpacked into `tools/` on Windows.

SPI Code's own tool processes get a scoped environment. The app's tools come first on PATH, followed by the PATH captured from the operator's login shell, as T3 Code does. That lets an app started from Finder find the same tools a terminal would. `KUBECONFIG` and `AZURE_CONFIG_DIR` point at app-owned folders, so `spi connect` and `az account set` never change the operator's own terminal setup. That means the operator signs in to Azure once inside SPI Code. Stray `GH_TOKEN` and `GITHUB_TOKEN` values are removed unless Settings names one. Sign-ins use each tool's own flow: `gh` through the browser (which also signs in Copilot), `glab` against `community.opengroup.org`, and `az login` with the stack's tenant filled in. Tokens stay in each tool's own store or the OS keychain. Commands in a code thread use the sandbox profile instead.

Look

SPI Code has one palette, Shore, in a light and a dark appearance. The appearance follows the system unless Settings pins one, and there is no palette picker. The token structure is T3 Code's semantic role model, cut down to the roles this app uses: canvas, chrome, sidebar, surface, raised, overlay, text, muted, accent, error, warning, success and code. Shore's values from Keelson's `design-tokens.ts` fill it. Data colors (status, series, priorities) are checked with Keelson's palette validator for contrast and color-blind separation in both appearances.

In the light appearance, sand (`#f3efe4`) is the chrome: title bar, left bar and right panel. The middle panel, where the reading happens, uses the near-white card color (`#fbf9f3`), so long text sits on the lightest surface. The teal accent marks the current selection, the primary button and the anchor chip.



Each swatch shows light, then dark. Sea lavender marks Copilot's own messages and proposals, so text the agent wrote is never confused with a fact the sweep read.

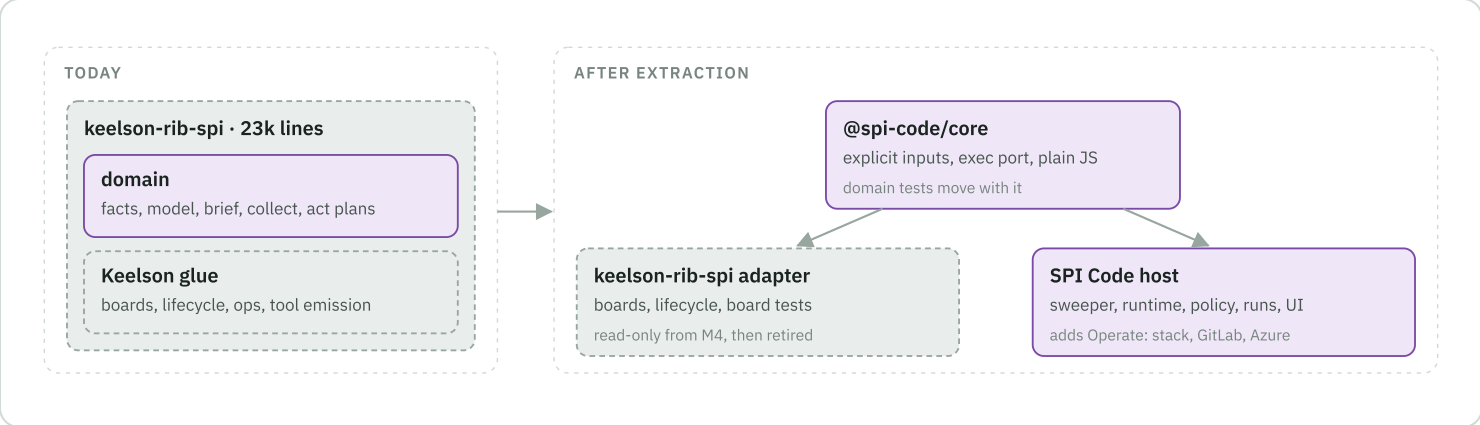
Relationship to Keelson

The SPI domain moves into `@spi-code/core`, a package in the SPI Code repository, in two steps. M1 moves the pure parts: Facts, the model, the briefing and `readItem`. M2 moves the collectors and action planning behind an exec port, once the host actually calls them.

The package has no Keelson, Bun or Electron dependency:

- Fleet membership, rollout policy, configuration and the clock are passed in as arguments, not read from module globals.
- `Bun.YAML` is replaced by `yaml`.
- Board tone types stop leaking into the model.
- The package ships compiled JavaScript.

The rib depends on a release tarball of core and becomes a thin adapter. It keeps its boards and Keelson lifecycle, so the Keelson tab keeps working.



The domain tests move into core and switch from `bun:test` to vitest, a mechanical change. The board and adapter tests stay in the rib. The M1 check is that all the moved tests pass and the live Keelson tab renders the same Today on the same sweep. Core lives in SPI Code's repository because, from M2 on, the host is what drives its changes. The rib pins a core version and bumps it only when asked. Two sweepers would share one GitHub GraphQL budget. So from M2, the rib's sweep is off by default and its tab reads the last Facts it recorded. Once M4 ships, the rib's verbs are switched off too, so two hosts can never pace or dispatch the same fleet action. The rib retires after SPI Code has been used daily for a release cycle. That decision is made then.

From Keelson, SPI Code takes the Shore tokens and the palette validator as code. It uses Keelson's Copilot provider as the reference for session handling and tool projection. It takes three ideas as concepts: verbs that return a client effect (open the inspector, open a run, start a thread), durable runs with cursor-read events, and the shapes of `investigate` and `plan-act-evaluate`. From T3 Code it copies, under MIT with the notice kept, the login-shell PATH capture, the verified app-owned installer, the readiness states, the semantic theme roles, the right-panel tab model and the electron-builder signing setup. It adapts T3 Code's work log, diff view, terminal drawer and composer approval strip for code threads.

Milestones

Each milestone ships something the operator uses. M0 and M1 run in parallel because neither depends on the other. Signing comes early, because hardened-runtime failures on the Copilot runtime only appear in a signed build. Windows is built and smoke-tested in CI from M0. Hands-on Windows operation starts at M7.

MILESTONE	SHIPS	PROVES BEFORE THE NEXT STARTS
M0 Spike	Electron shell, utilityProcess host, MessagePort contract, the SDK's platform runtime unpacked and started by path, one SPI tool gated in its handler, `node:sqlite`. Developer ID signed and notarized macOS arm64 build; Windows x64 CI build. An install-only probe of the Operate chain (uv, Python, `azure-cli` with pip, the `k8s-configuration` extension) on both platforms. A command sandbox probe on both platforms that runs `java -version` with the keychain and home config unreadable.	On a clean machine with no `~/.copilot`, the signed macOS build and the Windows build each sign in through gh, list models through the per-session token, start Copilot, stream a turn, register a stub for every tool name in appendix C and confirm that each one reaches the model and can be called, and refuse a denied call. A user-level `~/.copilot` MCP server is not visible to the session. The host model (utilityProcess or Node child) is chosen here and only that one is built, as is the signing approach for the SDK's runtime binaries: re-sign them with the app, or keep GitHub's signature.
M1 Core	`@spi-code/core` with Facts, model, briefing and `readItem`. The rib consumes its tarball.	The moved tests pass under vitest, and the live Keelson SPI tab renders the same Today on the same sweep.
M2 Watch	Collectors in core, sweeps in the host with a rate budget, program tree, the six console views, inspector, evidence envelope, Setup for Copilot, git, gh and glab.	On Facts recorded from one sweep, every view reads the same as the rib console. Then a day of live operation with no Keelson sweeping.
M3 Ask	Anchored threads, Copilot with the read tools, redaction, approval cards, run log, Diagnose a failure on Actions evidence.	On recorded incidents, Diagnose names the same cause as the inspector, and every claim cites a tool result.
M4 Act	The rib's eleven GitHub verbs plus `merge_pr`, `comment`, `open_issue` and `assign_coding_agent` as tiered tools with bindings and locks, intents and outcome lookups, durable waits, Hand off a fix. The rib's verbs are switched off.	A stale binding is refused in tests and live, a rerun-when-deployable wait survives a restart, and a write interrupted after sending is resolved by outcome lookup, not re-sent.

MILESTONE	SHIPS	PROVES BEFORE THE NEXT STARTS
M5 Code	Code threads: worktrees and the push lease, the host's worktree tools, the command sandbox on both platforms, Command cards, host commit and `push_branch`, the terminal drawer with node-pty rebuilt and signed, JDK and Maven in Setup, the PR watch after a push.	A sandboxed command cannot read the keychain, `~/.ssh` or `~/.kube`, cannot write the clone's git folder, and cannot get a planted hook or config value run by SPI Code's git. A push is refused when the remote moved. One real `fork_integration` fix goes from finding to green checks without leaving the app.
M6 Operate	Operate tools in Setup, environment dispatches, maintenance, reconcile and service pin and reset through `spi`, read tools through `kubectl` and `flux`, GitLab MR tools, live cluster evidence in the Stack view and in Diagnose.	One environment refresh and one service pin and reset run end to end, with every Consequential step carded. Live reads are labeled apart from inferred state.
M7 Team	Azure Trusted Signing on Windows, auto-update from GitHub releases, first-run welcome.	A second engineer installs from a release and reaches Watch without help.

Validation

Core keeps the rib's fixture tests. They cover attribution, incident folding, pacing, release naming and briefing wording, and run on every change. The host is tested against recorded Facts replays, so a sweep can be reproduced without touching GitHub.

The agent runtime is tested against a fake SDK session that scripts tool calls. Those tests check three things. Every tier answers the way the table says. A tool missing from `availableTools` cannot be reached. A policy error refuses the call.

Durability tests stop the host at each crash boundary: before an intent is written, after it is written but before the call, and after the call but before the result. Each test checks what the run reopens as.

Sandbox tests run commands that try to read each blocked path, write the clone's git folder, add a hook, set a config value and repoint the worktree's `.git` file, then check that SPI Code's commit and push ignore all of them. Push tests move the remote between card and approval and expect a refusal.

A packaged-app smoke test runs on macOS and Windows in CI. It starts the app, completes a scripted Copilot turn against a stub model, and screenshots each view in both appearances.

APPENDIX · REFERENCE MATERIAL

Reference detail

A · Repository and stack

PART	CHOICE
Repository	`spi-code`, a pnpm monorepo: · `apps/desktop`: Electron main and preload · `apps/window`: the renderer · `packages/core`: `@spi-code/core` · `packages/host`: sweeper, runtime, policy, runs, store · `packages/contracts`: zod message schemas · `packages/tools`: manifest and installers
Runtime and tests	Node as shipped in a pinned Electron. pnpm with a hoisted layout for electron-builder, as T3 Code stages it. vitest across the repository. Bun cannot load `node:sqlite`, so it is not used.

PART	CHOICE
Window	React 19, Vite, Tailwind v4, Base UI primitives wrapped as T3 Code wraps them, and Zustand stores fed by host events. No Effect, so zod is the one validation style from core to the window.
Store	`node:sqlite` in the host, with these tables: threads, messages, runs, run_events, intents, approvals, waits, worktrees, leases, facts_snapshots. Facts snapshots are kept for seven days. Settings are in `settings.json`.
Agent	`@github/copilot-sdk`, pinned. Its `@github/copilot-sdk-<platform>` runtime is listed in `asarUnpack` and started over stdio by explicit path. In-process mode is never used.
Packaging	electron-builder: dmg and zip on macOS arm64, NSIS on Windows x64. The SDK's runtime adds about 160 MB per architecture. Developer ID signing and notarization from M0. Azure Trusted Signing and electron-updater at M7.
Lint and format	Biome, matching the rib and Keelson.

B · Tool manifest

TOOL	GROUP	INSTALL	SIGN-IN
Copilot runtime	Watch	Inside the app (SDK platform package)	The gh OAuth token, read per session; needs a Copilot seat on that account. Setup recognizes a classic `ghp_` token named in Settings and explains that Copilot needs the gh OAuth sign-in.
git	Watch	macOS: Xcode command line tools prompt. Windows: PortableGit into tools/	gh as credential helper for SPI Code's own git; none inside the sandbox
gh	Watch	Release archive into tools/, checksum verified	`gh auth login --web`, repo and workflow scopes
glab	Watch	Release archive into tools/, checksum verified	Login against community.opengroup.org
JDK 17, Maven	Code	Release archives into tools/, checksum verified; one Maven local repository under app data	None

TOOL	GROUP	INSTALL	SIGN-IN
uv, Python	Operate	uv binary into tools/; uv-managed Python under app data	None
az	Operate	`uv tool install azure-cli --with pip`; `az extension add -n k8s-configuration`	`az login --tenant` with an app-owned `AZURE_CONFIG_DIR`
spi	Operate	uv tool install of the stack release wheel through the Microsoft package feed proxy, at the declared version	Uses az and the app kubeconfig
kubectl, kubelogin, flux, helm	Operate	Release binaries into tools/, checksum verified. kubectl is matched to the cluster's supported version skew	Azure session, app kubeconfig

C · Operator tools, tiers, bindings and outcome lookups

TIER	TOOLS	BINDING RE-READ BEFORE RUNNING	OUTCOME LOOKUP AFTER AN INTERRUPTION
Read	`spi_brief`, `spi_watchstand`, `spi_incidents`, `spi_line`, `spi_stack`, `spi_template`, `spi_prs`, `spi_issues`, `spi_security`, `spi_radar`, `spi_release_notes`, `read_artifact`, `run_log`, `pr_diff`, `mr_diff`, `stack_status`, `stack_info`, `flux_state`, `pod_logs`, `k8s_get`, `az_show`	None	None; a read is simply repeated

TIER	TOOLS	BINDING RE-READ BEFORE RUNNING	OUTCOME LOOKUP AFTER AN INTERRUPTION
Local	`read_file`, `search_files`, `edit_file`, `worktree_diff`, `commit` (code threads)	The path resolves inside the thread's worktree after links; a commit goes to the thread's own `spi/*` branch	A commit is applied if the branch head is the new commit
Command	`run_command` (code threads)	Worktree, folder and exact arguments on the card; the sandbox profile started	None; a command stopped by a restart is marked cancelled and needs a new card
Change	`push_branch`	Worktree clean with nothing untracked; remote at the expected commit; fast-forward; no `.github/` change unless the PR number is typed; PR open at the expected head; this thread holds the lease	Read the remote ref: at the pushed commit is applied; at the expected commit is not applied; anything else is unknown
Change	`approve_runs`	The run waits for approval; the PR is open and its head matches the held run	Applied if the run left the waiting state after the intent time
Change	`rerun`	Run id, attempt number, conclusion	Applied if the run's attempt number is higher than the bound one
Change	`merge_pr` (Dependabot or template PR, protected branch)	Head and base SHA, mergeable, required checks green, required reviews present, not draft; sent with `--match-head-commit`	Read the PR: merged with the bound head is applied; open, unmerged, at the bound head and with no merge queued is not applied; anything else is unknown
Change	`sync_template` (one fork), `sync_upstream` (prepare), `dispatch_cascade`, `rerun_cascade`, `fleet_release` (BOM only)	No run of the same workflow queued or running; template main SHA; the tracking issue is open	The run URL that `gh workflow run` returns is stored on the intent, and its run is read back. With no stored URL, a matching run by the operator after the intent time is shown as a candidate and the outcome stays unknown until the operator confirms

TIER	TOOLS	BINDING RE-READ BEFORE RUNNING	OUTCOME LOOKUP AFTER AN INTERRUPTION
Change	`open_issue`, `open_pr`, `open_upstream_mr`, `comment`	Target repository or thread exists and is open	Applied if an item carrying the intent marker exists
Change	`assign_coding_agent`	Issue open; not already assigned to the agent	Applied if the agent is among the issue's assignees
Consequential	`sync_all_forks`, `roll_out`, `start_upstream`	For each target: head SHA and checks, sent with `--match-head-commit` where it merges; pacing and ordering guards from core; lane capacity	Per target, using the merge and dispatch lookups above, in the run's target ledger
Consequential	`dispatch_env_refresh`, `dispatch_env_upgrade`, `dispatch_env_reset`	Tenant, subscription, cluster; declared version; no lifecycle run in flight; deployability evidence fresh, complete and in agreement	The dispatch lookup above, by the stored run URL, on the stack repository
Consequential	`stack_reconcile` (plain, refresh-images)	Cluster on the card; no lifecycle run in flight; variant on the card	Before the call, the intent records the source's reconcile annotation and, for refresh-images, the `osdu-image-lock` contents. Applied if the annotation now parses to a time after the intent (spi writes it in the host's local time) and differs from the recorded value; otherwise unknown. Reconcile is a trigger, so repeating it has the same effect: an unknown outcome offers a resend once the operator confirms
Consequential	`stack_reconcile` (resume)	Cluster on the card; the source is suspended; no lifecycle run in flight	Applied if the source reads not suspended; not applied if it still reads suspended
Consequential	`maintenance_set`	Cluster on the card; maintenance not set; no lifecycle run in flight	Applied if `spi status` reports maintenance set

TIER	TOOLS	BINDING RE-READ BEFORE RUNNING	OUTCOME LOOKUP AFTER AN INTERRUPTION
Consequential	`maintenance_clear`	No lifecycle run in flight; deployed ref matches the declared stack version; `spi status` reports ready, maintenance set, and maintenance as the only blocker. Deployability is not required	Applied if `spi status` reports maintenance clear
Consequential	`service_pin`, `service_reset`	Cluster, service, digest, current pin owner; deployability evidence fresh, complete and in agreement	Applied if the live digest and pin owner match the intent

When a lookup itself fails, the outcome is unknown and retry stays disabled. Every dispatch and creation lookup is limited to items created by the operator's own account.

D · Copilot session checklist

BEHAVIOR	HANDLING
Disconnect is asynchronous	Wait for the previous session to release before resuming it, as Keelson's `waitForSessionRelease` does. Resuming early closes the new session.
Resume ignores model and effort	Call `session.setModel` after every resume.
Warm clients are shared	Count references, so a retry on one thread never stops a client another thread is streaming on.
Connection errors	Match the SDK's connection errors, restart the runtime and resume the session; the in-flight tool call is marked cancelled.
Operator's own Copilot setup	`empty` mode and an app-owned base directory; `availableTools` on every session.
Sign-in under empty mode	Empty mode sees no existing login and would store a new one in plain text. Each session takes its token from a provider that reads `gh auth token`, so an account switch is followed. The provider API is experimental and is proved in M0. The fallback is a client-level token with the warm client dropped on a credential change. Setup checks the account has a Copilot seat with a real session probe.
Thread kinds	`availableTools` is set from the thread's kind when the session starts. A code thread's session includes the worktree tools; an operate thread's never does.

BEHAVIOR	HANDLING
Tool deferral	Register every SPI tool with `defer: "never"`; with no built-in tool search, a deferred tool cannot be reached.
SDK upgrades	Run the fake-session suite and the packaged smoke test on both platforms before the pin moves.

E · After version 1

ITEM	SHAPE
Drive a PR to green	Playbook: read checks and threads, classify failures with core attribution, re-run Environment-class failures once, hand code fixes to the coding agent, wait for CI, repeat up to a round limit. The merge stays its own card.
Upgrade the stack	Playbook: declaration diff, bump PR, wait for its merge, `env-upgrade` card, follow maintenance, verify running revision, gateway and smoke, draft the consumer note.
Provisioning and data writes	`spi up` (including `--dry-run`, which creates a resource group), `spi down`, Bicep, entitlement changes and data loads. First provisioning needs a binding that expects the environment to be absent, which v1 does not define.
Container builds	A container engine for local `spi build` and paired tests in a worktree.
macOS x64 and release-notes PDF	The SDK ships a darwin-x64 runtime, so Intel Macs are packaging work when someone needs them. A PDF of release notes would use `webContents.printToPDF` on a hidden window.
Thread-wide approvals	Letting one approval cover repeats of a Change tool on one target within a thread. Added only if per-card approval proves tiring.

F · Alternatives not chosen

ALTERNATIVE	WHY NOT
Fork T3 Code and strip it	About 650k lines built for many providers and remote use, with heavy upstream churn and Effect throughout. Every package is private, so nothing can be depended on. The strip would touch contracts, settings and onboarding across the tree and leave conceptual weight the app never uses.

ALTERNATIVE	WHY NOT
Tauri or a Swift and WinUI app	The domain, the Copilot SDK and the rib's tests are TypeScript on Node. Tauri would still need a Node sidecar for them, and two native UIs would double the window work.
Copilot through ACP, as T3 Code does	Copilot CLI accepts client tools over ACP only as an MCP HTTP server. The SDK registers tools in the host, with no server to secure.
Host as a localhost WebSocket server	Only needed for remote clients. A MessagePort gives the same separation with no port and no token.
Port Keelson's YAML workflow engine	It is bound to Bun, and its approval pause does not survive a restart, the property playbooks need most. Two typed state machines on the run store are smaller than the port.
A general command tool	A wrapper, interpreter or script defeats any classification by binary and verb. Typed tools with explicit variants are the only way the tier table can be enforced. Code threads run commands only inside the sandbox, one approved card each, with no credentials.
Copilot built-in file reads with path limits	The callback does see read paths, but a path allowlist for Copilot's own reader would be a second file policy beside the tools. `read_artifact` keeps one boundary: only files the host saved for the run.
Extract core inside the rib repository	From M2, the host drives core's changes. Keeping core in the rib would put every host change behind a rib release.
Copy the domain into the app and retire the rib now	Loses the live tab as the check that nothing changed, and leaves two diverging copies.
Code changes only through the coding agent	The agent opens PRs against main, so it cannot take `fork_integration` conflicts or template-owned files. Those fixes would leave the app for a terminal.
A command allowlist by name	Once a command starts, it and its child processes run whatever they like, so a name check proves nothing. The sandbox and the credential-free environment hold for every process a command starts.
Commands with the operator's environment	Default config files, the keychain and the SSH agent would give every approved build the operator's GitHub, Azure and cluster access.
Merging cascade PRs in the app	Gate 2 is a human review on GitHub. `merge_pr` covers Dependabot and template PRs only.
Homebrew and WinGet installs	A clean Mac has no Homebrew, and WinGet can need elevation. uv covers Python, az and spi on both platforms without either.
Several palettes with a picker	One palette in two appearances covers the need. The role model leaves room for more later at no cost now.

G · Risks

RISK	HANDLING
Copilot runtime in a signed, packaged app	Proved in M0 on a notarized build before anything depends on it. A Node child process is the fallback for the host.
SDK shape changes	Pinned version, fake-session suite, Keelson's provider as a second reference.
GitHub API budget	Core keeps its batching. The host adds a budget that slows sweeps before the GraphQL limit, and Setup shows the points left.
Coding agent cannot do the work	Handoff is offered only for changes on the fork's main branch. The issue records the evidence, so the operator can take it over in a code thread.
Command sandbox on Windows	Seatbelt profiles are proven on macOS by other agent tools. The Windows restricted-token profile is probed in M0 and proved in M5. Without a working sandbox, code threads offer no commands.
Data leaving through an approved command	Network stays on for Maven. Each card shows the exact command, the sandbox holds no credentials to send, and Settings states the limit.
Rib and app both acting	The rib's verbs are switched off at M4. Both pin the same core version, and the rib carries no domain logic.